

Machine Learning for Business

GTSC2143

Lecture 7 Classification II

Instructor: Dr. Lily Bei Luo

AY2025-2026 Semester 1

Review: Predicting sentiment by topic

Predicting sentiment by topic

Intelligent restaurant review system

All reviews for restaurant

★★★★★ 7/21/2015
This is probably my favorite place to eat Japanese in Seattle. My boyfriend and I ordered nigiri of scallop, Japanese snapper (seasonal), and the agedashi tofu and 2 special rolls. I would skip the special rolls, because the nigiri and sashimi cuts is where this place excels. The tofu, as recommended by other Yelpers was amazing. It's more chewy and the sauce/gravy is the perfect amount of flavor for the delicate tofu.

★★★★★ 6/11/2015
Dining here at the sushi bar made me feel like sitting front row to an amazing performance. We didn't have resos, banged down to the ID after work, got here breathlessly at 5:10pm, and got the last two seats in the place.

★★★★★ 6/9/2015
I came here having high expectations due to the reviews of this place, but I was bit disappointed. The restaurant is small so do make reservations when you come here. Dishes cost from \$4-26 each and dishes are small.

Break all reviews into sentences

The seaweed salad was just OK, vegetable salad was just ordinary.

I like the interior decoration and the blackboard menu on the wall.

All the sushi was delicious.

My wife tried their ramen and it was pretty forgettable.

The sushi was amazing, and the rice is just outstanding.

The service is somewhat hectic.

Easily best sushi in Seattle.

Sentence
Sentiment
Classifier

Average
predictions

Sushi
★★★★★

Most
👍 & 👎

Easily best
sushi
in Seattle.

Predicting sentiment by topic

Representing the sentences

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
corpus = [
```

```
    'This is the first document.',
```

```
    'This document is the second document.',
```

```
    'And this is the third one.',
```

```
    'Is this the first document?',
```

```
]
```

```
vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(corpus)
```

```
vectorizer.get_feature_names_out()
```

```
>>>array (['and', 'document', 'first', 'is', 'one', 'second', 'the', 'third',  
          'this'],...)
```

```
print(X.toarray())
```

```
>>>[[0 1 1 1 0 0 1 0 1]
```

```
 [0 2 0 1 0 1 1 0 1]
```

```
 [1 0 0 1 1 0 1 1 1]
```

```
 [0 1 1 1 0 0 1 0 1]]
```

- Fit: Analyzes the text in the training data (X_train) to learn the vocabulary (i.e., all the unique words in the training set) and their corresponding feature indices.
- Transform: Converts the training text into a word count matrix based on the learned vocabulary.

Discussion Question: How to do this task?

How to do this task?

Image classification

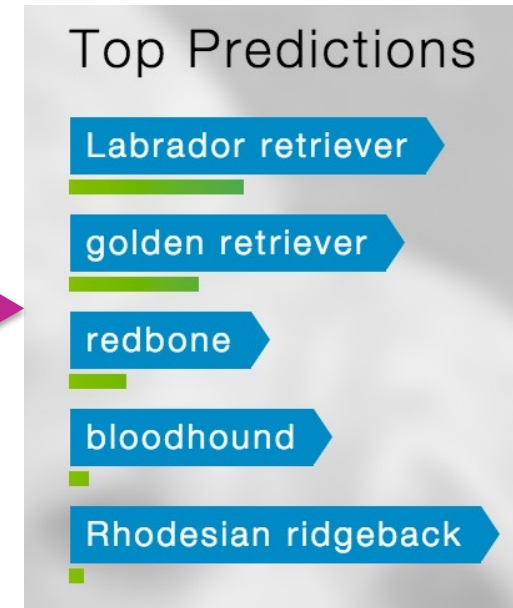
Question:

- How would you classify a dog image?
- What's the difference between text and image data?



The inputs are
image pixels

Input: x
Image pixels



Output: y
Predicted object

How to do this task?

Personalized medical diagnosis

Input: x



The input is
multimodal data

Question:

- How to integrate multimodal data (e.g., numerical, image, text, etc.)?

Output: y

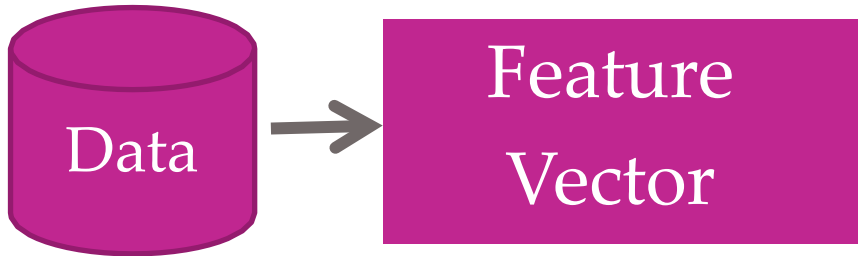
Disease
Classifier
MODEL

Healthy
Cold Flu
Pneumonia
...

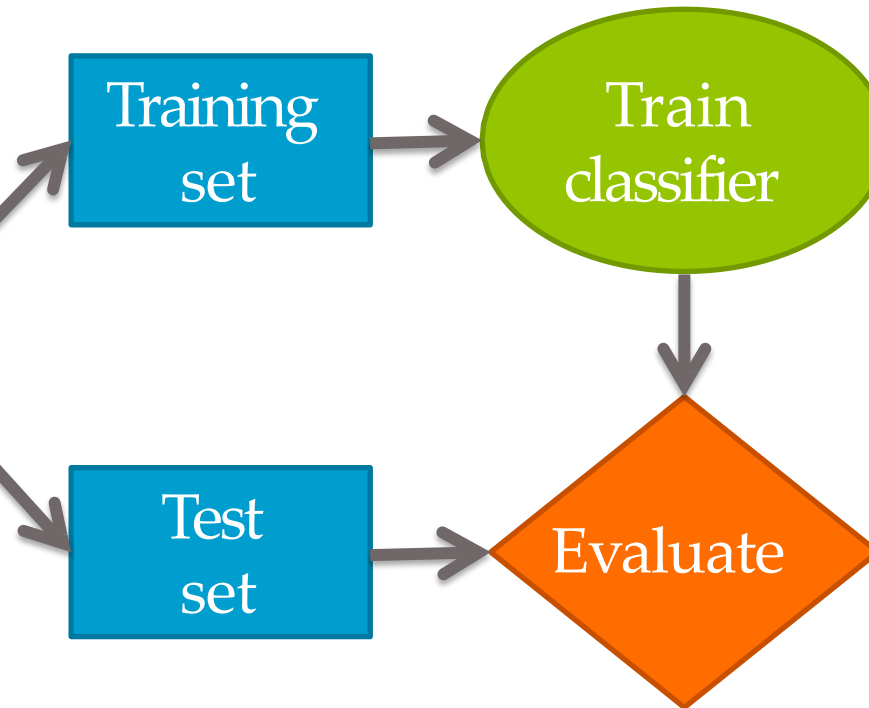


General Framework for Classification Tasks

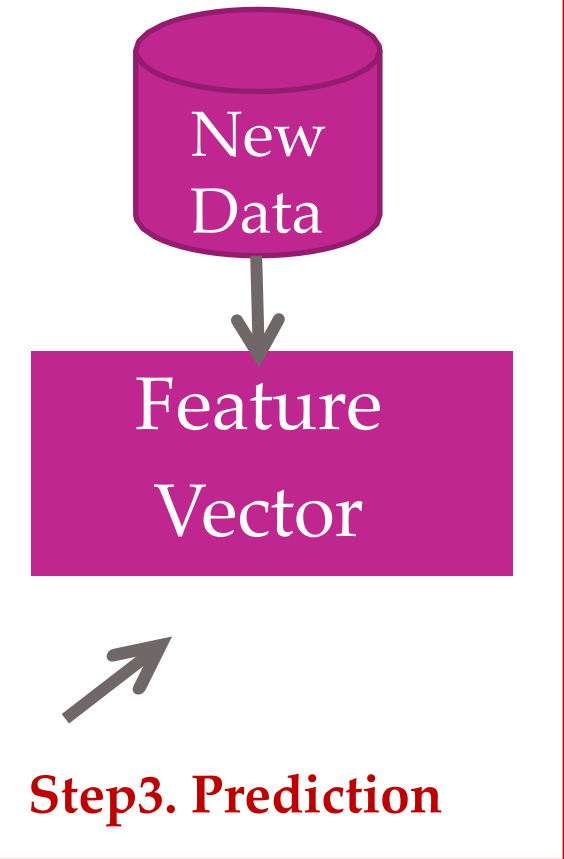
Step1. Feature Representation



Step2. Model Training



Step3. Prediction



Feature Representation

From Data to Features

Feature representation is the process of converting raw data into a format that is suitable for machine learning or statistical analysis. Raw data is often in unstructured formats such as images, text, and audio, while machine learning algorithms typically require fixed-length numerical vectors as input. Feature representation **establishes the mapping from raw data to numerical vectors**, where each dimension represents a specific attribute or feature of the data.

- **Numerical data** can be used directly, but usually require normalization or standardization to ensure that different feature scales and units do not affect algorithm performance.
- **Categorical data** require encoding. One-hot encoding converts each category into a binary vector; label encoding maps categories to integers
- **Text data** commonly use Bag of Words, TF-IDF, word embeddings (Word2Vec, GloVe), or representations from pre-trained language models (BERT, GPT).
- **Image data** can be hand-crafted features (such as SIFT, HOG) or feature representations automatically learned by deep learning models.

Feature Representation

Numerical Data

Data normalization is a preprocessing method that **resizes the range of feature values to a specific scale**, usually between 0 and 1. It is a feature scaling technique used to transform data into a standard range. Normalization ensures that features with different scales or units contribute equally to the model and improves the performance of many machine learning algorithms.

1. Min-Max Normalization

- Min-Max normalization rescales a feature to a specific range, typically $[0, 1]$

$$x_{normalized} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

2. Logarithmic Transformation

- Log transformation compresses large values and spreads out small values

$$x_{normalized} = \log(x + 1)$$

3. Z-score Normalization (Standardization)

- Z-score normalization (Standardization) centers the data around zero and scales it to unit variance

$$x_{normalized} = \frac{x - \mu}{\sigma}$$

Feature Representation

Categorical Data

One Hot Encoding is a method for converting categorical variables into a binary format. It creates new columns for each category where 1 means the category is present and 0 means it is not. The primary purpose of One Hot Encoding is to ensure that categorical data can be effectively used in machine learning models.

Fruit	Categorical value of fruit	Price
apple	1	5
mango	2	10
apple	1	15
orange	3	20

Fruit_apple	Fruit_mango	Fruit_orange	Price
1	0	0	5
0	1	0	10
1	0	0	15
0	0	1	20

How One-Hot Encoding Works: An Example

Feature Representation

Categorical Data

Label Encoding is a method for converting categorical variables into numerical format by assigning a unique integer to each category. It maps each distinct category to a sequential number, typically starting from 0.

Level	Level_coded
low	0
medium	1
high	0

Understanding Label Encoding

Categorical data is broadly divided into two types:

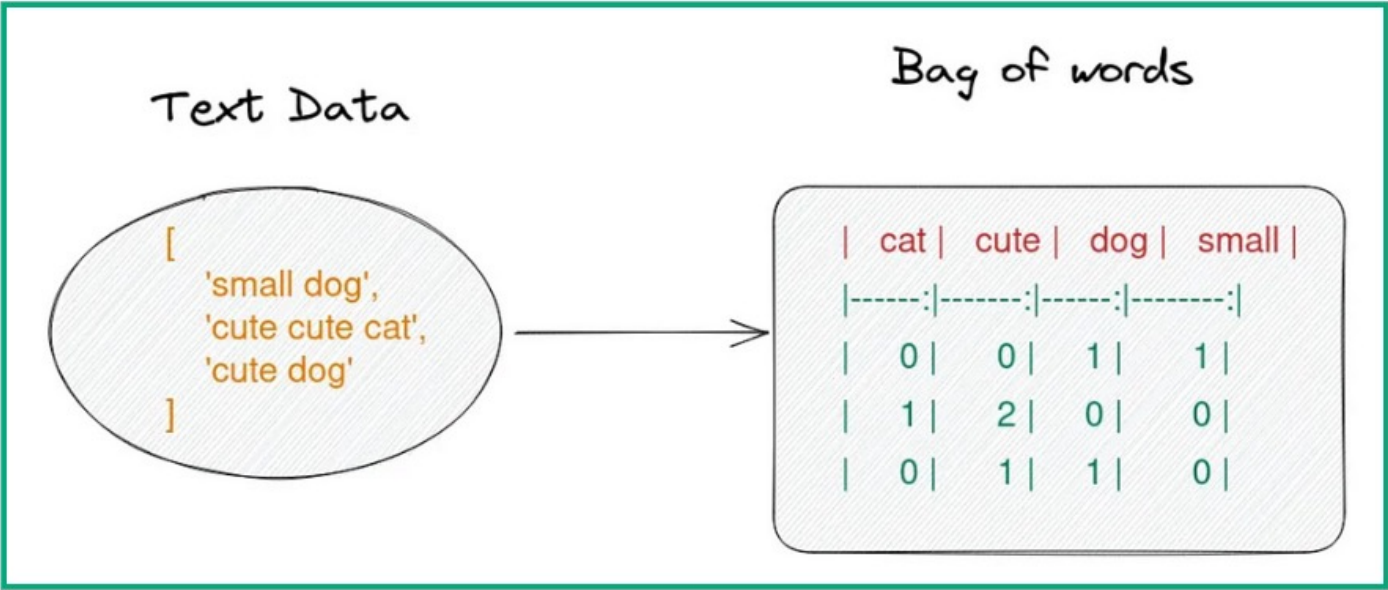
- **Nominal Data:** Categories without inherent order (e.g., colors: red, blue, green).
- **Ordinal Data:** Categories with a natural order (e.g., satisfaction levels: low, medium, high).

Label encoding works best for ordinal data, where the assigned numbers reflect the order.

Feature Representation

Text Data

Bag-of-Words (BoW) is a text representation technique that represents a document as an unordered set of words and their respective frequencies. It discards the word order and captures the frequency of each word in the document, creating a vector representation.



Feature Representation

Text Data

Bag-of-Words (BoW) is a text representation technique that represents a document as an unordered set of words and their respective frequencies. It discards the word order and captures the frequency of each word in the document, creating a vector representation.

Advantages

- Simplicity and explainability
- Ease of implementation
- Scalability

Disadvantages

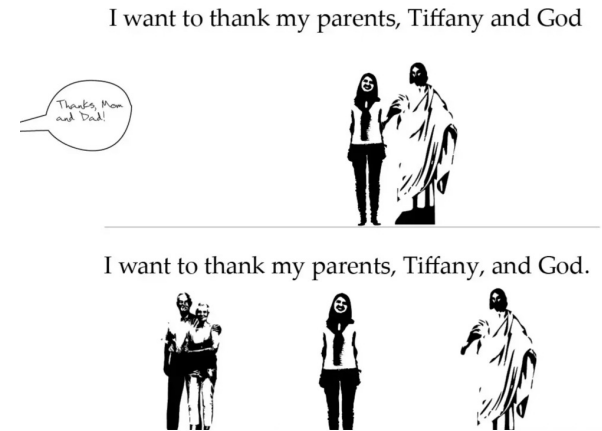
- Insensitivity to word order
- Insensitivity to grammatical structure
- Limited semantic information

Document D1	<i>The child makes the dog happy</i> the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	<i>The dog makes the child happy</i> the: 2, child: 1, makes: 1, dog: 1, happy: 1



	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

Two documents with different meanings, yet same BoW representation



Because of Punctuation, these two sentences mean different things, yet get represented by same BoW vectors (Source: [https://www.quora.com/What-is-the-difference-between-Bag-of-Words-and-Tfidf-Vectorizer](#))



© AIML.com Research

The baseball **pitcher** drank a **pitcher** of water

Feature Representation

Text Data

Term Frequency-Inverse Document Frequency, commonly known as TF-IDF, is a numerical statistic that reflects the importance of a word in a document relative to a collection of documents (corpus). It is widely used in natural language processing and information retrieval to evaluate the significance of a term within a specific document in a larger corpus. TF-IDF consists of two components:

- **Term Frequency (TF):** Measures how often a term (word) appears in a document.
- **Inverse Document Frequency (IDF):** Measures the importance of a term across a collection of documents.

$$TF(t, d) = \frac{\text{number of times } t \text{ appears in } d}{\text{total number of terms in } d}$$

$$IDF(t) = \log \frac{N}{1 + df}$$

$$TF - IDF(t, d) = TF(t, d) * IDF(t)$$

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	1/7	1/7	$\log(2/2) = 0$	0	0
Car	1/7	0	$\log(2/1) = 0.3$	0.043	0
Truck	0	1/7	$\log(2/1) = 0.3$	0	0.043
Is	1/7	1/7	$\log(2/2) = 0$	0	0
Driven	1/7	1/7	$\log(2/2) = 0$	0	0
On	1/7	1/7	$\log(2/2) = 0$	0	0
The	1/7	1/7	$\log(2/2) = 0$	0	0
Road	1/7	0	$\log(2/1) = 0.3$	0.043	0
Highway	0	1/7	$\log(2/1) = 0.3$	0	0.043

Feature Representation

Text Data

Word2Vec is a neural approach for generating word embeddings. It belongs to the family of neural word embedding techniques and specifically falls under the category of distributed representation models. It is a popular technique in natural language processing (NLP).

- Represent words as continuous vector spaces.
- Capture the semantic relationships between words by mapping them to high-dimensional vectors.

There are two neural embedding methods for Word2Vec:

- **Continuous Bag of Words (CBOW)**
- **Skip-gram**

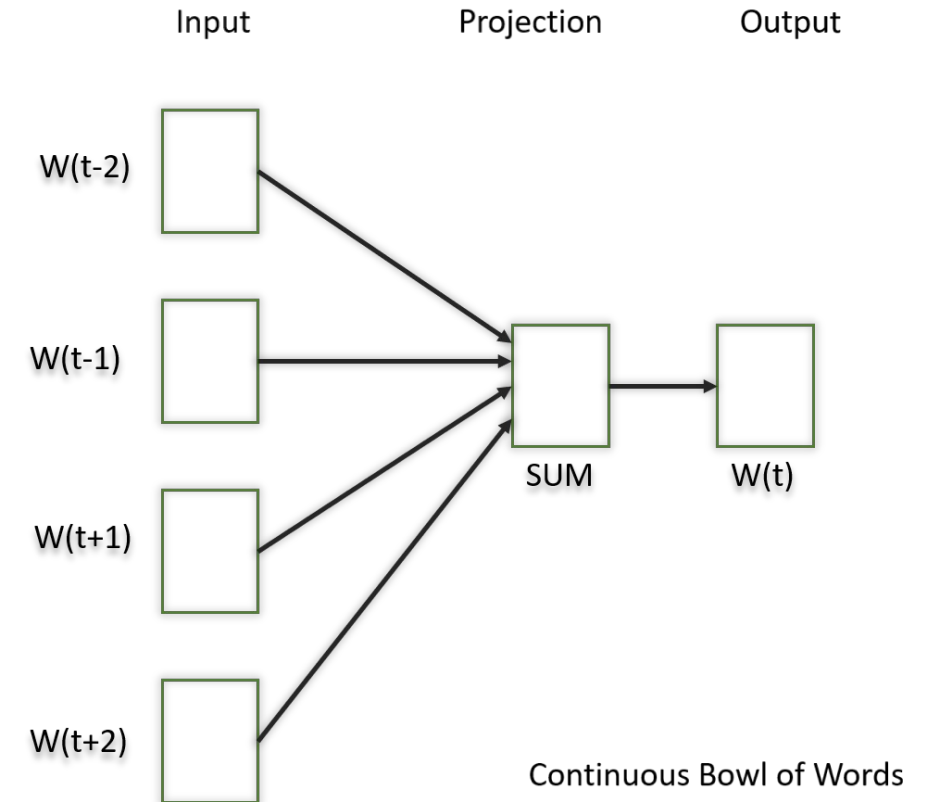
Feature Representation

Text Data

Continuous Bag of Words (CBOW) predicts a target word based on its surrounding context words. It takes multiple words around the target word as input and trains the model to predict the center target word. Specifically, given a window size, CBOW takes several words before and after the target word, averages or sums their word vectors as input, and then predicts the center word.

Characteristics:

- CBOW trains faster and works better for frequently occurring words since it gathers information from multiple context words to predict the target word.
- It's suitable for smaller datasets.



Feature Representation

Text Data

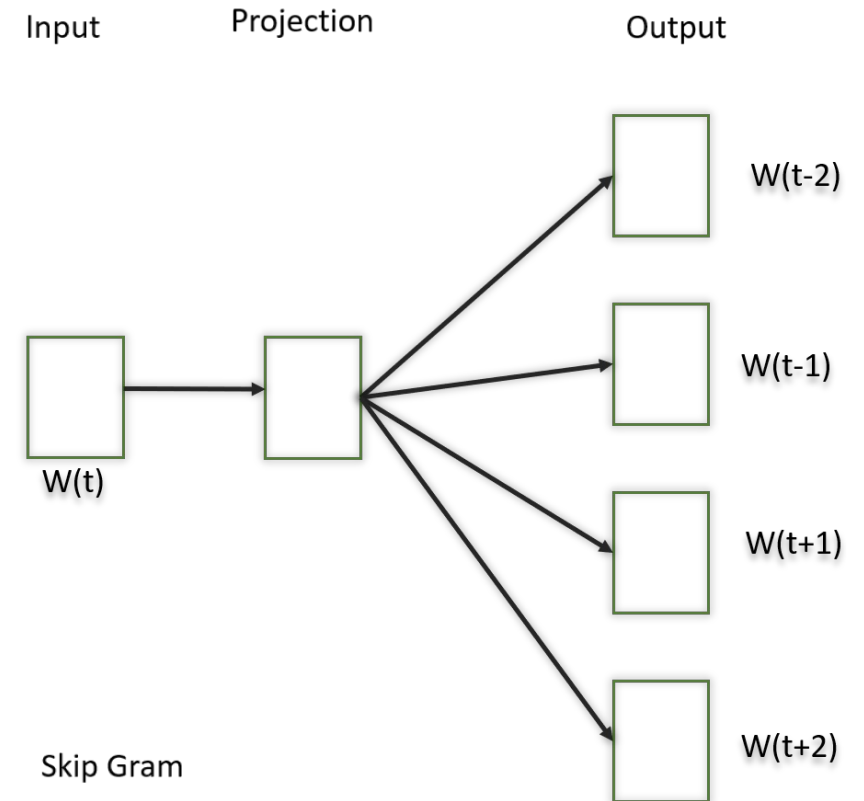
Skip-gram works in the opposite way to CBOW. **It uses a target word to predict the words in its context.** Given a center word, the model attempts to predict all words within the surrounding window.

Characteristics:

- Skip-gram performs better with rare words because even low-frequency words are used as input to predict multiple context words, giving them more training opportunities.
- It typically produces higher-quality word vectors but requires longer training time.

Generally,

- choose Skip-gram when you have a large dataset and care about the representation quality of rare words;
- choose CBOW when you have a smaller dataset and prioritize training efficiency.



Feature Representation

Text Data

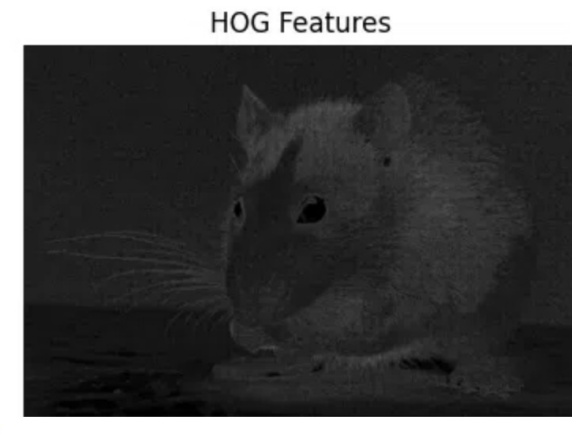
Pre-trained Word Embeddings are representations of words that are learned from large corpora and are made available for reuse in various Natural Language Processing (NLP) tasks. These embeddings capture semantic relationships between words, allowing the model to understand similarities and relationships between different words in a meaningful way.

1. **GloVe** is trained on global word co-occurrence statistics. It leverages the global context to create word embeddings that reflect the overall meaning of words based on their co-occurrence probabilities.
2. **FastText** extends Word2Vec by representing words as bags of character n-grams. This approach is particularly useful for handling out-of-vocabulary words and capturing morphological variations.
3. **Bidirectional Encoder Representations from Transformers (BERT)** is a transformer-based model that learns contextualized embeddings for words. It considers the entire context of a word by considering both left and right contexts, resulting in embeddings that capture rich contextual information.

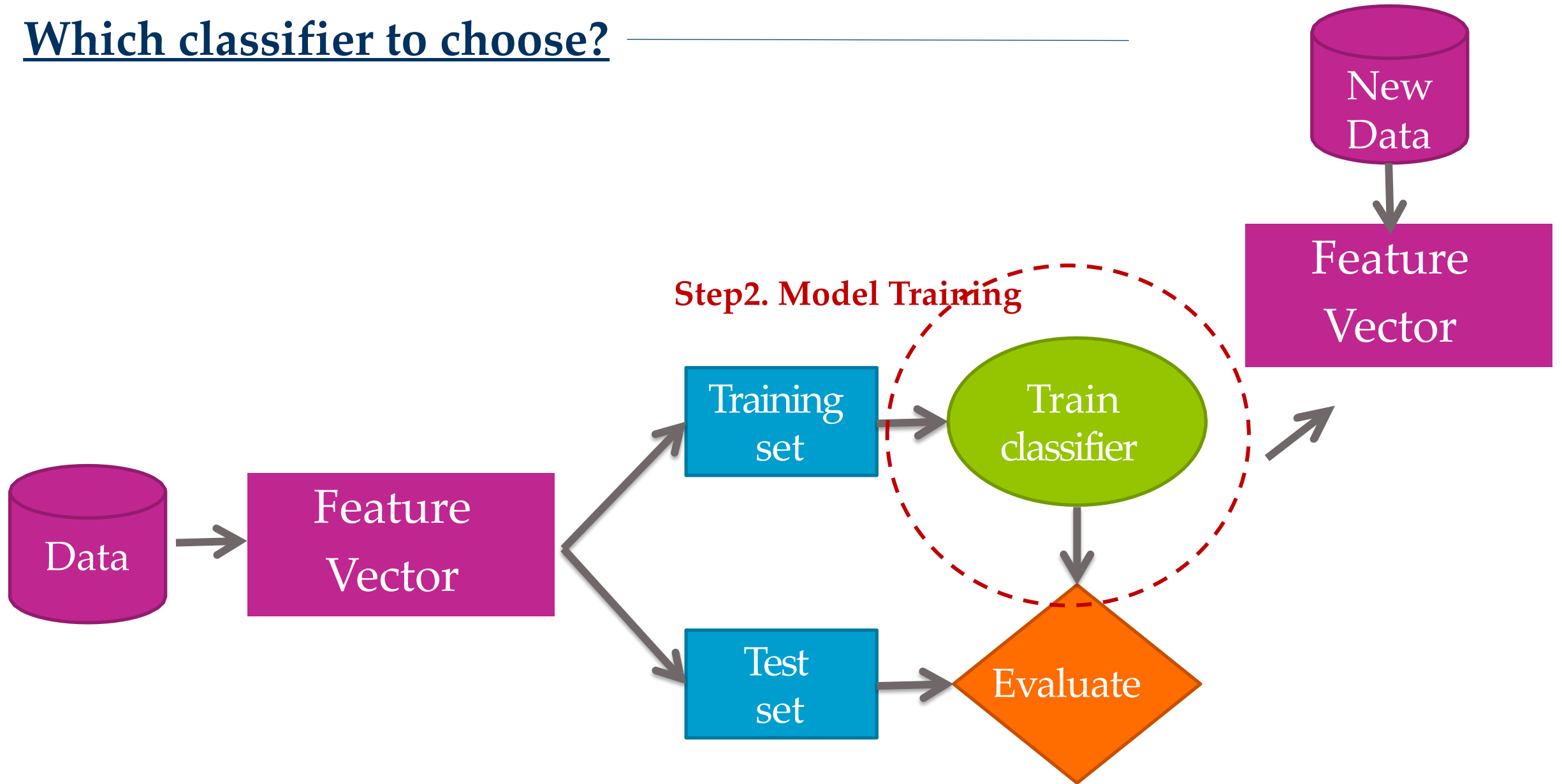
Feature Representation

Image Data

- **(Scale-invariant Feature Transform) SIFT** describes local image regions by their gradient magnitudes and orientations. It is known for its robustness to various transformations making it more effective for use. It basically extracts the key points of the reference image and store it in a database. An object is recognized in a new image by individually comparing each feature to the database created.
- **Histogram of Oriented Gradients (HOG)** captures the distribution of gradient orientations in an image region, making it useful for detecting objects with varying textures and shapes. It is valued for its ability to capture the shape and texture information of objects in a way that is robust to changes in illumination and viewpoint.



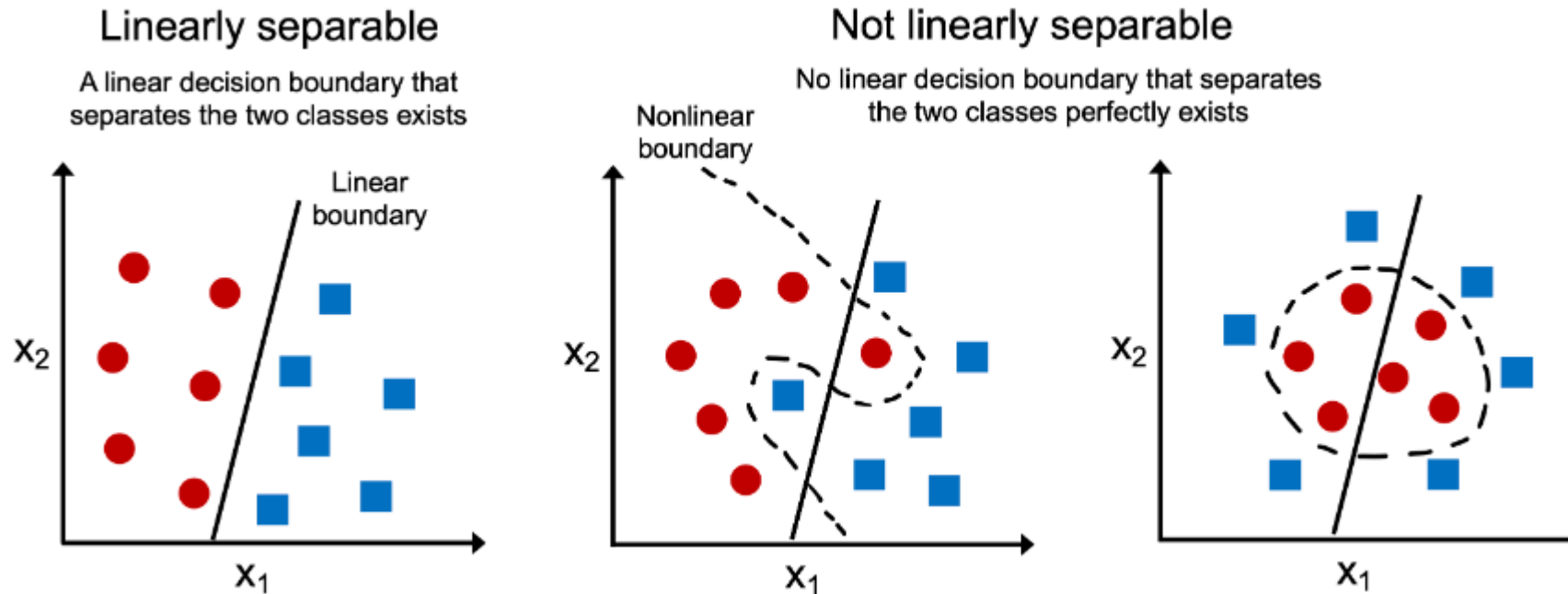
Which classifier to choose?



Classifier

Linear vs Non-Linear Classification

- **Linear Classification** refers to categorizing a set of data points to a discrete class based on a linear combination of its explanatory variables.
- **Non-Linear Classification** refers to separating those instances that are not linearly separable.



Classifier

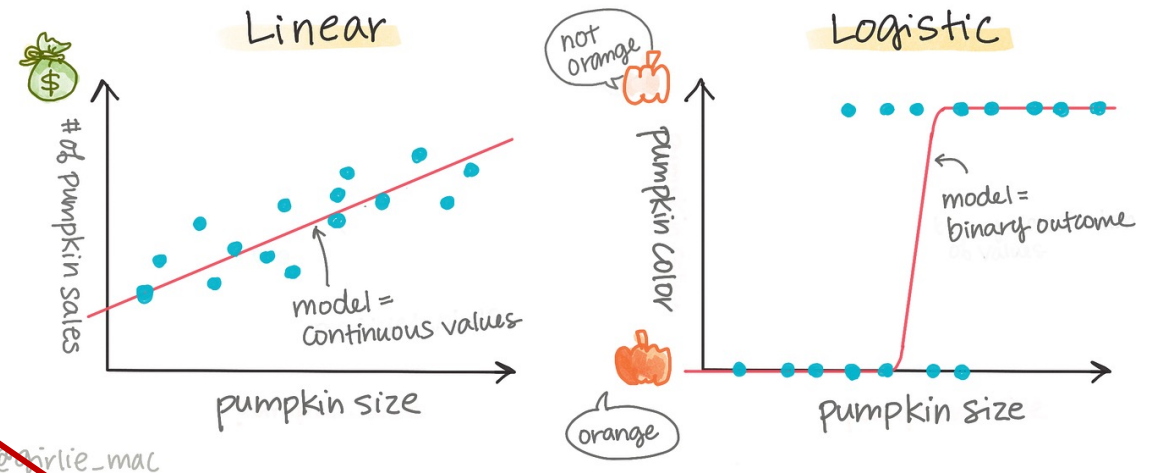
Linear Classifier

Logistic Regression is one of the most fundamental and widely used algorithms in machine learning for classification tasks. Unlike linear regression which predicts continuous values it predicts the probability that an input belongs to a specific class.

- **Binomial Logistic Regression:** This type is used when the dependent variable has only two possible categories, like Yes/No, Pass/Fail or 0/1.
- **Multinomial Logistic Regression:** This is used when the dependent variable has three or more possible categories that are not ordered, like "cat," "dog" or "sheep."
- **Ordinal Logistic Regression:** This type applies when the dependent variable has three or more categories with a natural order or ranking, like "low," "medium" and "high."

Most commonly used

LINEAR vs. LOGISTIC REGRESSION

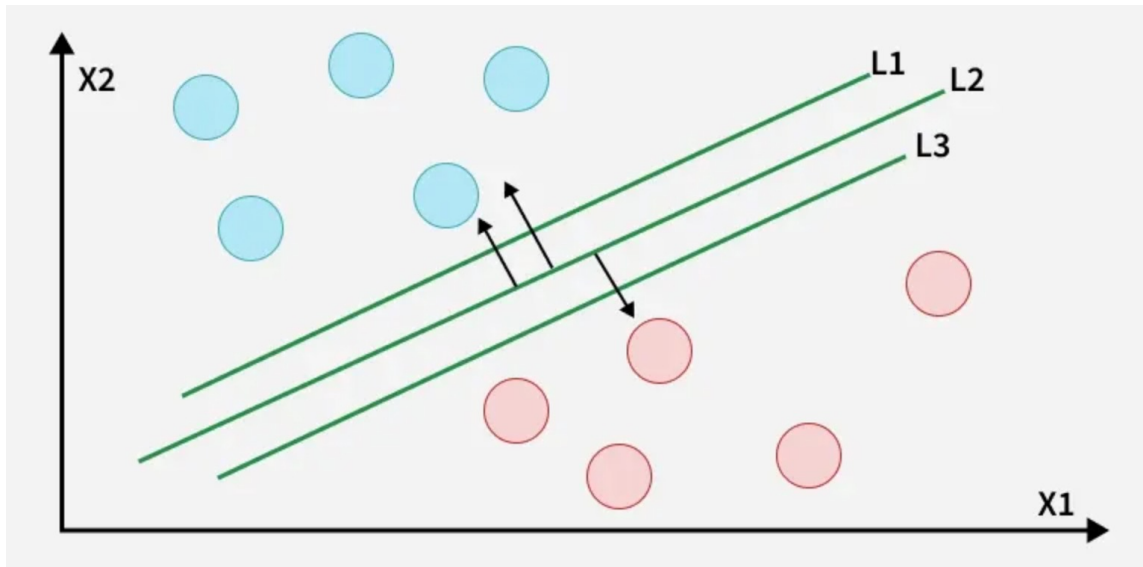


Extensions of the logistic function

Classifier

Linear Classifier

Support Vector Machine (SVM) finds a hyper-plane that creates a boundary between the various data types. The key idea behind the SVM algorithm is to find the hyperplane that best separates two classes by maximizing the margin between them. This margin is the distance from the hyperplane to the nearest data points (support vectors) on each side.



The best hyperplane also known as the "**hard margin**" is the one that maximizes the distance between the hyperplane and the nearest data points from both classes. This ensures a clear separation between the classes.

So from the left figure, we choose L2 as hard margin.

Classifier

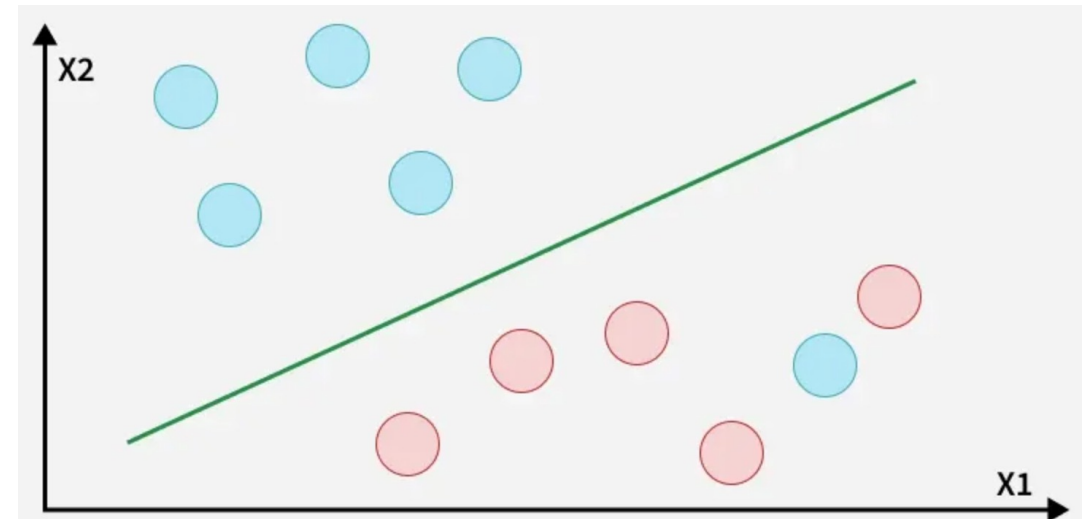
Linear Classifier

Support Vector Machine (SVM) optimizes the following equation to balance margin maximization and penalty minimization:

$$\text{Objective Function} = \left(\frac{1}{\text{margin}} \right) + \lambda \sum \text{penalty}$$

The penalty used for violations is often hinge loss which has the following behavior:

- If a data point is correctly classified and within the margin there is no penalty (loss = 0).
- If a point is incorrectly classified or violates the margin the hinge loss increases proportionally to the distance of the violation.



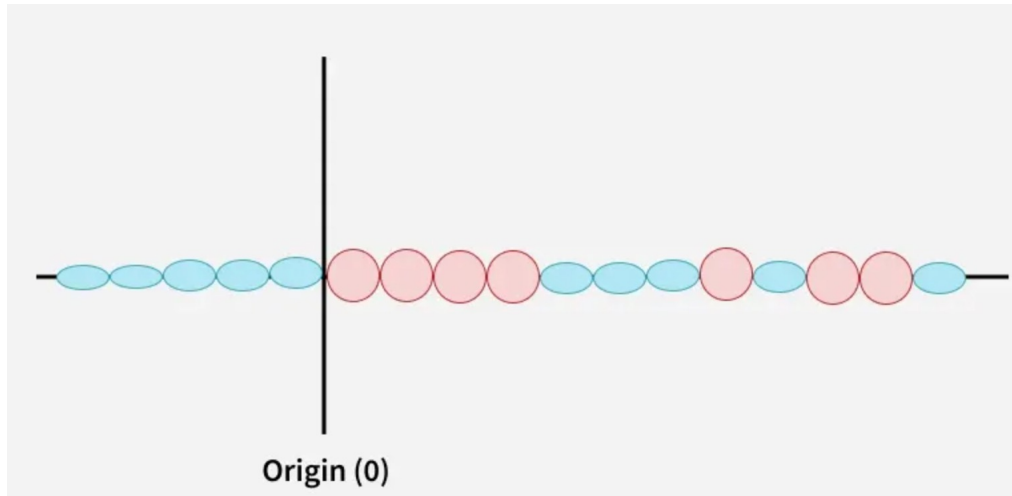
The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin.

Classifier

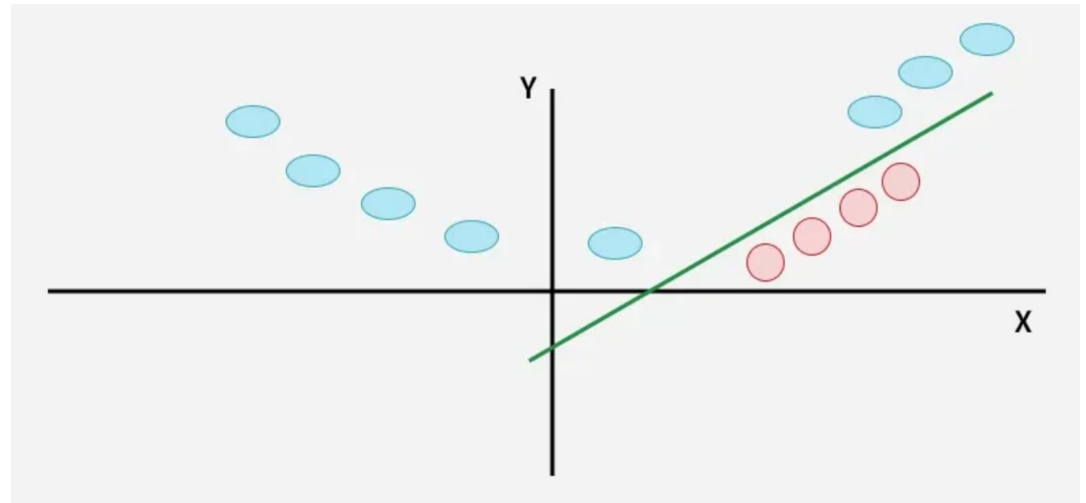
Nonlinear Classifier

Support Vector Machine (SVM) uses a technique called **kernels to map the data into a higher-dimensional space where it becomes separable** when data is not linearly separable i.e it can't be divided by a straight line. This transformation helps SVM find a decision boundary even for non-linear data.

- A kernel is a function that maps data points into a higher-dimensional space without explicitly computing the coordinates in that space.



Original 1D dataset for classification



Mapping 1D data to 2D to become able to separate the two classes

Classifier

Nonlinear Classifier

Naive Bayes predicts the category of a data point using probability. It assumes that all features are independent of each other. Naive Bayes performs well in many real-world applications such as spam filtering, document categorization and sentiment analysis.

- It is named as "Naive" because it assumes the presence of one feature does not affect other features. The "Bayes" part of the name refers to its basis in Bayes' Theorem.

$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

Labels for the equation above:

- $P(c|x)$: Posterior Probability
- $P(x|c)$: Likelihood
- $P(c)$: Class Prior Probability
- $P(x)$: Predictor Prior Probability

$$P(c|X) = P(x_1|c) \times P(x_2|c) \times \dots \times P(x_n|c) \times P(c)$$

Outlook	Temp	Humidity	Windy	Play
Rainy	Cool	High	True	?

$$P(\text{Yes} | X) = P(\text{Rainy} | \text{Yes}) \times P(\text{Cool} | \text{Yes}) \times P(\text{High} | \text{Yes}) \times P(\text{True} | \text{Yes}) \times P(\text{Yes})$$

$$P(\text{Yes} | X) = 2/9 \times 3/9 \times 3/9 \times 3/9 \times 9/14 = 0.00529 \rightarrow 0.2 = \frac{0.00529}{0.02057 + 0.00529}$$

$$P(\text{No} | X) = P(\text{Rainy} | \text{No}) \times P(\text{Cool} | \text{No}) \times P(\text{High} | \text{No}) \times P(\text{True} | \text{No}) \times P(\text{No})$$

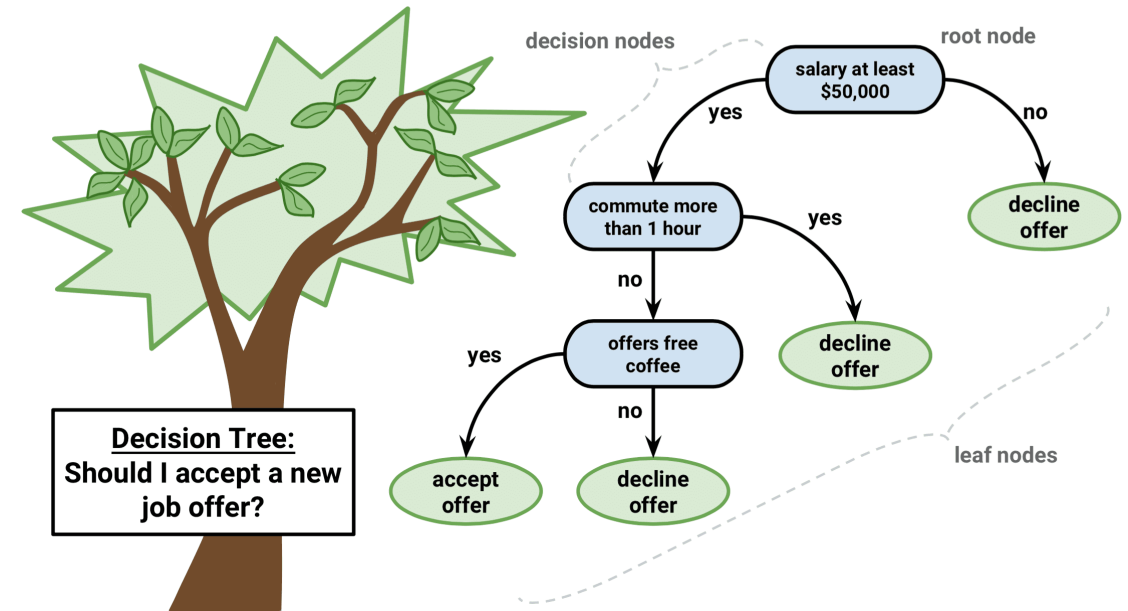
$$P(\text{No} | X) = 3/5 \times 1/5 \times 4/5 \times 3/5 \times 5/14 = 0.02057 \rightarrow 0.8 = \frac{0.02057}{0.02057 + 0.00529}$$

Classifier

Nonlinear Classifier

A **Decision Tree** helps us make decisions by showing different options and how they are related. It has a tree-like structure that starts with one main question called the root node which represents the entire dataset. From there, the tree branches out into different possibilities based on features in the data.

- **Root Node:** Starting point representing the whole dataset.
- **Branches:** Lines connecting nodes showing the flow from one decision to another.
- **Internal Nodes:** Points where decisions are made based on data features.
- **Leaf Nodes:** End points of the tree where the final decision or prediction is made.



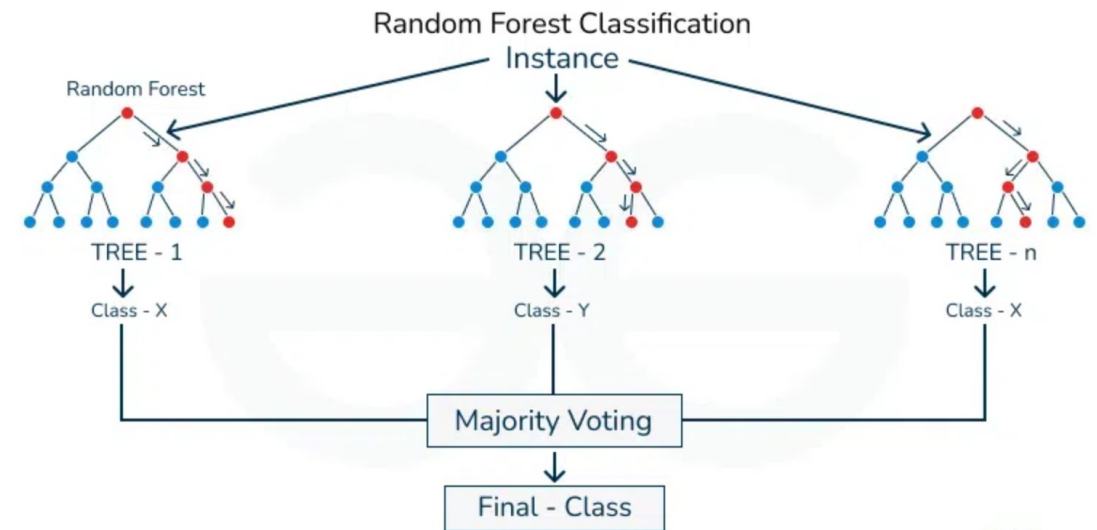
Classifier

Nonlinear Classifier

Random Forest combines the predictions of multiple decision trees to produce a more accurate and stable result. It can be used for both classification and regression tasks. In classification tasks, it uses multiple decision trees and outputs the label that has the maximum votes among all the individual tree predictions.

Working of Random Forest Classifier

1. **Bootstrap Sampling:** Random rows are picked (with replacement) to train each tree.
2. **Random Feature Selection:** Each tree uses a random set of features (not all features).
3. **Build Decision Trees:** Trees split the data using the best feature from their random set. Splitting continues until a stopping rule is met (like max depth).
4. **Make Predictions:** Each tree gives its own prediction.
5. **Majority Voting:** The final prediction is the one most tree agree on.



Classifier

Nonlinear Classifier

K-Nearest Neighbors (KNN) works by finding the "k" closest data points (neighbors) to a given input and makes a predictions based on the majority class (for classification) or the average value (for regression). Since KNN makes no assumptions about the underlying data distribution it makes it a non-parametric and instance-based learning method.

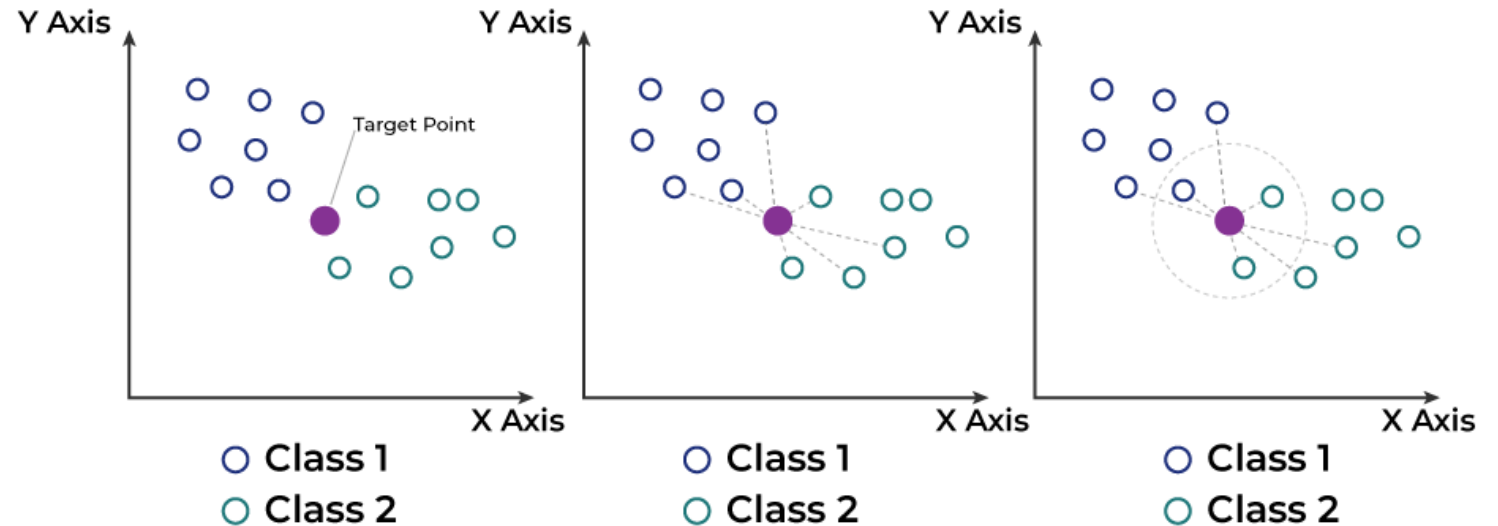
Working of KNN algorithm

Step 1: Selecting the optimal value of K

Step 2: Calculating distance

Step 3: Finding Nearest Neighbors

Step 4: Voting for Classification or
Taking Average for Regression

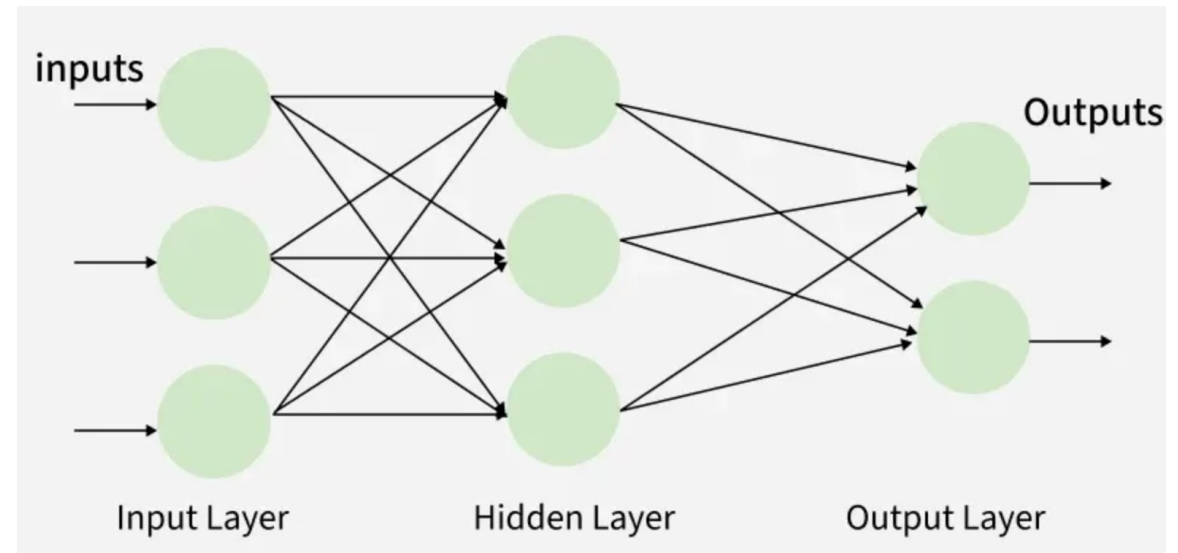


Classifier

Nonlinear Classifier

Multi-Layer Perceptron (MLP) consists of fully connected dense layers that transform input data from one dimension to another. It is called multi-layer because it contains an input layer, one or more hidden layers and an output layer. The purpose of an MLP is to model complex relationships between inputs and outputs.

- **Input Layer:** Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers:** MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer:** The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.



Classifier Selection Guidelines

1. Understand your problem and data

- Define the problem: Is it a binary or multiclass classification?
- Analyze your data: Consider the size of your dataset (large vs. small), its complexity (linear vs. non-linear relationships), and the number of features.
- Consider constraints: Do you need a highly interpretable model? Do you have time or computational resource limitations?

2. Experiment with different classifiers

- Try a variety of algorithms: It's often best to test multiple algorithms, as one may perform better than others for a specific task.

3. Evaluate and refine your models

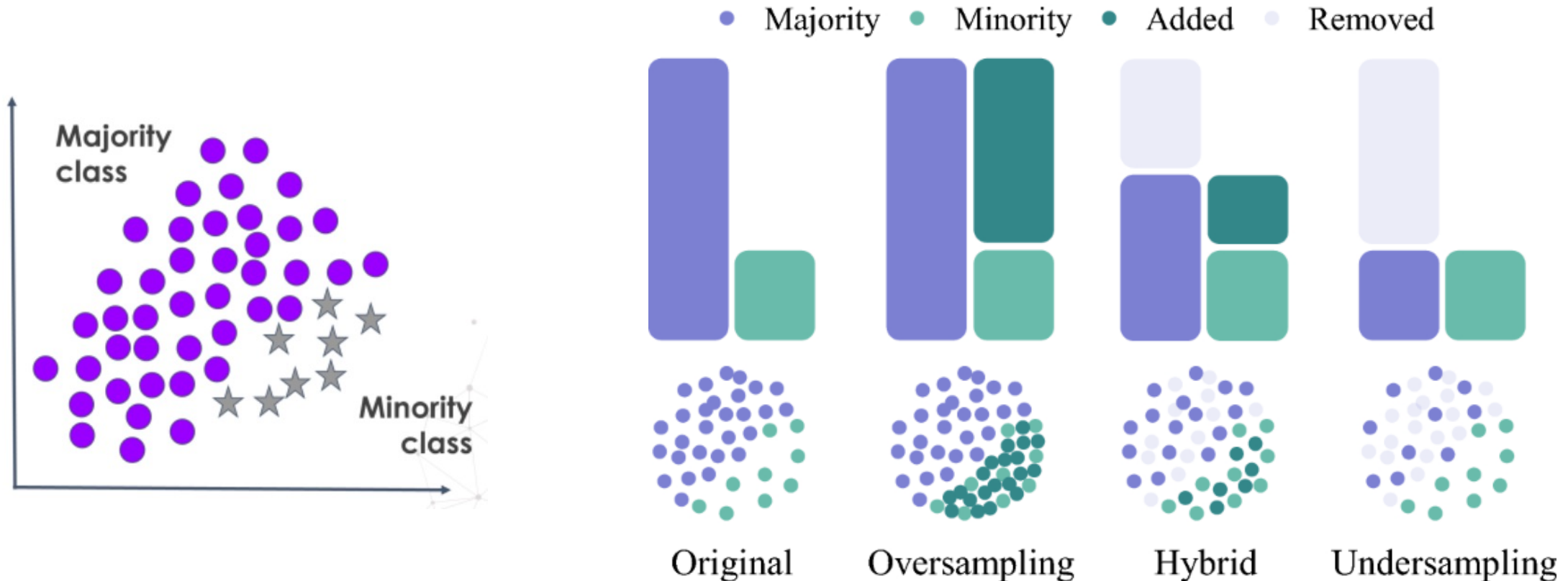
- Use cross-validation
- Compare performance

4. Consider simplicity and future steps

- Favor simpler models
- Keep exploring

Data Imbalance in Classification Problem

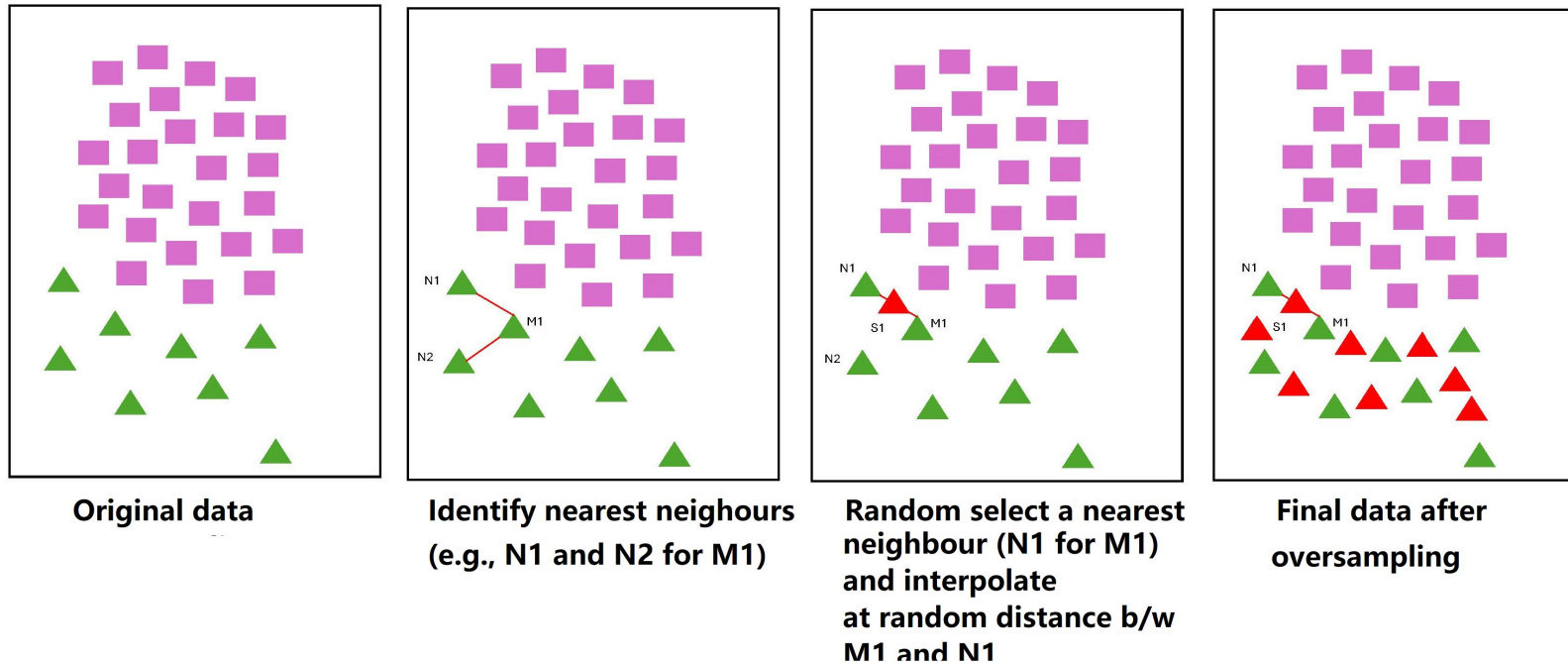
Data imbalance in classification refers to skewed class distribution, hindering machine learning models' performance. Majority classes dominate while minority classes are underrepresented. This challenge arises when one category vastly outnumbers others.



Data Imbalance in Classification Problem

The **Synthetic Minority Over-Sampling Technique (SMOTE)** is a powerful method used to handle class imbalance in datasets. SMOTE handles this issue by generating samples of minority classes to make the class distribution balanced. SMOTE works by generating synthetic examples in the feature space of the minority class.

SMOTE Process for Oversampling Minority Classes



Python Implementation

Python Implementation

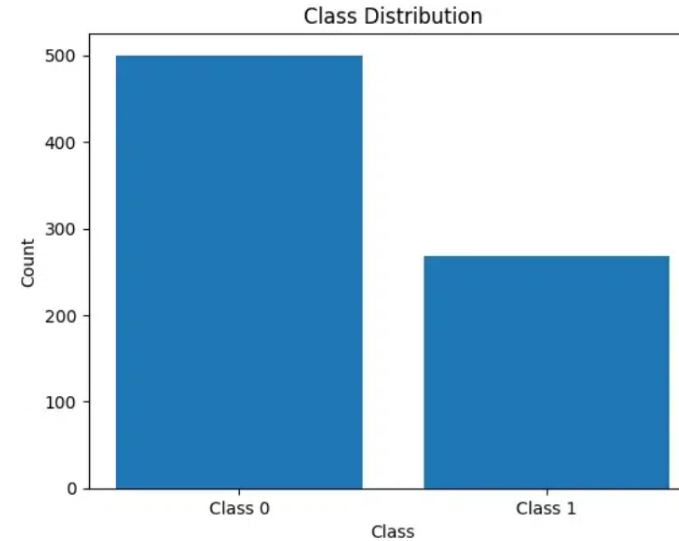
SMOTE for Imbalanced Classification

```
from imblearn.over_sampling import SMOTE

smote=SMOTE(sampling_strategy='minority')
x,y=smote.fit_resample(x,y)
y.value_counts()

>Outcome
1      500
0      500
Name: count, dtype: int64
```

Distribution after SMOTE



Distribution before SMOTE

Python Implementation

Classifiers

Most classification models are available in scikit-learn and can be implemented in **just a few lines of code**.

- https://scikit-learn.org/stable/user_guide.html

`DecisionTreeClassifier` is a class capable of performing multi-class classification on a dataset.

As with other classifiers, `DecisionTreeClassifier` takes as input two arrays: an array X, sparse or dense, of shape `(n_samples, n_features)` holding the training samples, and an array Y of integer values, shape `(n_samples,)`, holding the class labels for the training samples:

```
>>> from sklearn import tree
>>> X = [[0, 0], [1, 1]]
>>> Y = [0, 1]
>>> clf = tree.DecisionTreeClassifier()
>>> clf = clf.fit(X, Y)
```

Take decision tree as an example

Tutorial

